分布式 MiniSQL 最终设计报告

大规模信息系统构建技术导论

2026 年 4 月 27 日

浙江大学 计算机科学与技术学院

目录

I. 引言 4
I.1. 项目目标 4
I.2. 设计说明 4
I.3. 系统实现概览 4
I.4. 技术路线 5
II. 总体设计 6
II.1. 系统总体架构设计 6
II.2. ZooKeeper 元数据结构 7
II.3. RPC 接口设计 7
II.4. CAP 理论权衡 8
III. 模块与功能详细设计 9
III.1. Master 设计 9
领导者选举 9
DDL 编排与元数据更新 9
负载均衡 9
Region 迁移 10
路由修复与故障恢复 10
III.2. RegionServer 设计 11
启动与注册 11
MiniSQL 进程管理 11
SQL 执行路径 11
WAL 管理 12
副本同步 12
数据迁移与文件校验 13
III.3. Client 设计 13
SQL 分类与路由 13
路由缓存与失效 13
重试与读降级 14
路由指标 14
III.4. C++ MiniSQL 存储引擎 14
IV. 详细功能设计 16
IV.1. 建表流程 16
IV.2. 写入流程 16
IV.3. 查询流程 17
IV.4. Master 故障转移 17
IV.5. RegionServer 故障与恢复 17
IV.6. Region 迁移流程 17
V. 关键实现难点与处理方式 19
V.1. 元数据发布顺序 19
V.2. 写入确认顺序 19
V.3. 迁移期间的数据可见性 19

V.4. 客户端路由失效处理 20
V.5. 测试对实现边界的确认 20
VI. 系统测试与验证 21
VI.1. 自动化测试体系 21
VI.2. 系统测试结果 21
VI.3. 验证结论 22
VII. 系统功能实现情况 23
VII.1. 核心功能 23
VII.2. 工程质量与测试保障 24
VIII. 总结 25

I. 引言

I.1. 项目目标

本项目是在单机 MiniSQL 基础上实现的分布式关系型数据库课程项目。系统保留 C++ MiniSQL 作为本地存储与 SQL 执行引擎，在其外层实现 Java 分布式服务层，提供表级数据分布、三副本复制、Master 高可用、ZooKeeper 集群协调、客户端路由缓存、WAL、迁移恢复与混沌测试能力。

本系统可以完成分布式环境下的基本关系型数据库操作，并在节点故障、主节点切换和网络异常等测试场景中按 CP 契约处理读写请求。系统采用表级 Region 粒度，每张表作为一个完整分布单元分配到多个 RegionServer，避免行级分区带来的 RowKey、Split、Merge 与跨 Region 聚合复杂度，同时保留分布式数据库最关键的设计内容：元数据一致性、副本同步、故障感知、路由更新和恢复过程。

I.2. 设计说明

本程序采用 C++ 与 Java 两种程序设计语言实现。C++ 部分负责 MiniSQL 单机存储引擎，Java 部分负责分布式服务层、RPC 通信、ZooKeeper 协调和客户端访问。项目使用 Maven 组织 Java 多模块工程，使用 Docker Compose 编排完整集群，使用 Apache Thrift 定义跨进程 RPC 接口。

具体程序由 5 人组成的小组开发而成。小组成员的具体分工如表所示：

成员姓名	学号	分工
师东祺	3230101967	分布式通信与 Region Server 框架
周子安	3230103169	分布式通信与 Region Server 框架
徐浩然	3230105281	一致性协议与容灾
李浩博	3230101962	一致性协议与容灾
李业	3230102276	Master 高可用与负载均衡

I.3. 系统实现概览

层次	最终实现
存储引擎	C++ MiniSQL，包含 Catalog、Record、Index、Buffer、Interpreter、WAL 与 Crash Recovery

分布式协调	ZooKeeper 三节点集群，保存 Master 选主、RegionServer 注册、表路由和副本分布
Master	多 Master 高可用、元数据管理、DDL 编排、路由修复、负载均衡与迁移调度
RegionServer	管理 MiniSQL 进程，执行 SQL，维护 WAL，处理副本同步、数据迁移和 HTTP 状态输出
Client	交互式 REPL，SQL 路由，缓存失效，重试，读降级，路由指标输出
RPC	Apache Thrift 定义 4 组跨模块服务接口
测试	C++ 单测、Java 单测与集成测试、Docker 集群测试、压力测试、混沌测试、端到端 SQL 测试用例

I.4. 技术路线

本系统采用“C++ 存储内核 + Java 分布式控制层”的架构。C++ MiniSQL 已经包含 B+ 树索引、Buffer Manager、Record Manager、Catalog Manager 和 WAL，适合作为每个 RegionServer 的本地存储引擎。Java 层通过进程管理组件管理其生命周期和标准输入输出，对外提供分布式路由、复制、迁移和容错能力。

Java 负责 ZooKeeper、Thrift RPC、副本协议、Master 选主、Region 迁移和路由缓存；C++ 负责单机 SQL 语义、页缓存、索引与本地持久化。这一方案让系统可以同时验证数据库内核和分布式协调机制。

II. 总体设计

II.1. 系统总体架构设计

本系统最终部署形态为 10 个容器：

组件	数量	职责
ZooKeeper	3	提供命名空间 `/supersql`，保存 Master、RegionServer、表元数据与副本分布
Master	3	通过 ZooKeeper 选出一个 Active Master，其余为 Standby
RegionServer	3	每个节点运行 Java 服务与一个本地 MiniSQL 进程
Client	1	提供交互式 SQL REPL 和路由指标接口

系统整体访问路径如下：

图 1 — 系统总体架构图

如图 1 所示，Client 不经过 Master 转发数据读写，而是根据表路由直接访问 RegionServer；Master 主要负责控制面，ZooKeeper 负责选主和元数据协调。

请求类型	路径
建表	Client -> Active Master -> 多个 RegionServer 执行 `CREATE TABLE` -> 写 ZooKeeper 元数据
删表	Client -> Active Master -> 所有副本 RegionServer 执行 `DROP TABLE` -> 删除 ZooKeeper 元数据
写入	Client -> 路由缓存或 Master 查询 -> 主副本 RegionServer -> WAL -> 副本同步 -> MiniSQL 执行
查询	Client -> 路由缓存或 Master 查询 -> RegionServer -> MiniSQL 执行 -> Client

迁移	Master -> RegionAdminService 暂停写入、传输文件、更新元数据、清理源副本

II.2. ZooKeeper 元数据结构

系统所有业务节点都位于 `/supersql` 命名空间下。主要目录如下：

路径	节点类型	用途
`/masters`	LeaderLatch 节点	Master 选主与在线 Master 感知
`/active-master`	持久节点	保存当前 Active Master 的 `epoch`、`masterId`、`address` 和时间戳
`/region_servers/{rsId}`	临时节点	RegionServer 注册与心跳，包含地址、表数量、QPS、CPU、内存和副本决议信号
`/meta/tables/{table}`	持久节点	表路由元数据，包含主副本、全部副本、状态和版本号
`/assignments/{table}`	持久节点	表到 RegionServer 副本列表的分配关系

ZooKeeper 在系统中承担两类职责。第一类是控制面协调，包括 Master 选主、Active Master 发布和 RegionServer 上下线感知。第二类是元数据存储，包括表路由、表状态和副本列表。RegionServer 通过临时节点上报心跳，Master 通过 Watcher 感知 membership 变化，并触发 route repair 与 rebalance。

II.3. RPC 接口设计

系统用 Thrift 描述全部跨进程接口，生成 Java 代码后供 Master、RegionServer、Client 使用。最终定义了 4 组服务：

服务	调用方向	核心方法
`MasterService`	Client -> Master	`getTableLocation`，`createTable`，`dropTable`，`listTables`，`listRegionServers`，`triggerRebalance`

RegionService	Client -> RegionServer	execute , executeBatch , createIndex , dropIndex , ping
RegionAdminService	Master -> RegionServer	pauseTableWrite , resumeTableWrite , transferTable , copyTableData , deleteLocalTable , invalidateClientCache
ReplicaSyncService	RegionServer -> RegionServer	syncLog , pullLog , getMaxLsn , commitLog , finalizeLogDecision , getLogDecisionState

状态码统一在 StatusCode 中定义，包括 OK 、 ERROR 、 TABLE_NOT_FOUND 、 TABLE_EXISTS 、 RS_NOT_FOUND 、 REDIRECT 、 MOVING 、 NOT_LEADER 、 REPLICA_TIMEOUT 。这使客户端可以区分普通失败、路由错误、迁移重试和 Master 重定向。

II.4. CAP 理论权衡

系统按 CP 思路设计：优先保证数据正确性和分区容忍能力，在网络分区或副本不足时允许写入失败。在三副本部署下，RegionServer 写路径默认要求至少 1 个从副本 ACK，即主副本加一个从副本形成 2/3 多数派后才执行本地 MiniSQL 写入并提交 WAL。若可见副本数或 ACK 数不足，写请求返回明确错误，而不是降低副本确认门槛。

读路径提供两种语义。默认 EVENTUAL 模式允许 SELECT 在主副本不可达时访问副本，提高故障期间可读性；STRONG 模式只访问主副本，避免读到副本追赶前的旧数据。系统按 CP 契约处理故障场景：写成功后通过 WAL 和副本决议机制推动副本一致；副本确认不足时，写请求返回明确错误。

III. 模块与功能详细设计

III.1. Master 设计

主服务器 Master Server 是系统控制面的核心，负责选主、表元数据、DDL 编排、负载均衡、迁移、故障恢复和状态观测。

领导者选举

Master 选举模块使用 Curator `LeaderLatch` 在 `/masters` 下参与选举。获得领导权的 Master 会通过 CAS 更新 `/active-master`，将 `epoch` 加一并写入当前 `masterId` 与 `address`。客户端和 Standby Master 都以 `/active-master` 作为 Active Master 的事实来源。

Master 对非主请求采取拒绝策略。`createTable`、`dropTable` 和 `triggerRebalance` 在非 Active Master 上返回 `NOT_LEADER`，并带上 `redirectTo`。这避免 Standby 节点在选主异常、网络抖动或旧进程恢复后继续处理写控制命令。

DDL 编排与元数据更新

具体来讲，建表流程由 Active Master 统一完成：

1. 解析 `CREATE TABLE` 中的表名。
2. 检查 ZooKeeper 可用性和表是否已存在。
3. 按负载均衡策略选择最多 3 个 RegionServer。
4. 依次向所有目标副本发送完整 DDL。
5. 全部副本建表成功后，写入 `/meta/tables/{table}` 和 `/assignments/{table}`。
6. 若副本建表或元数据写入失败，Master 会删除已创建副本并删除已写入的元数据，避免出现半创建状态。

删表流程由 `dropTable` 完成。Master 会对 assignment 中的所有副本发送 `DROP TABLE`。只有所有副本返回成功或 `TABLE_NOT_FOUND`，才删除 ZooKeeper 元数据；如果存在失败副本，保留元数据并返回失败副本列表，便于再次执行删除操作。

负载均衡

负载计算由 Master 的负载均衡模块提供。最终评分公式以表数量为主信号，同时加入 QPS、CPU 和内存作为排序因素：

```
loadScore = tableCount * 100.0 + qps1min + cpuUsage + memUsage
```

Master 建表时选择评分最低的节点作为副本候选。定时 rebalance 由 `RebalanceScheduler` 触发，也可通过管理接口手动触发。默认配置中，调度周期为 30 秒，均衡阈值为 1.5 倍；若最繁忙节点的负载没有超过平均负载乘阈值，则跳过迁移。

Region 迁移

迁移逻辑由 Master 的迁移模块统一处理。迁移使用显式状态机表示过程：

状态	含义
`ACTIVE`	表可正常读写
`PREPARING`	准备迁移，写入迁移上下文，准备暂停写请求
`MOVING`	数据正在传输，写请求返回 `MOVING`
`FINALIZING`	数据传输已完成，正在更新 assignment 并清理源副本
`ROLLBACK`	迁移失败，正在恢复原始元数据
`COMPENSATING`	补偿清理需要继续确认，保留上下文供恢复逻辑继续处理

迁移步骤如下：

1. Master 将表状态写为 `PREPARING`，记录 `migrationAttemptId`、source 和 target。
2. 通过 `RegionAdminService.pauseTableWrite` 暂停主副本写入。
3. 将状态推进到 `MOVING`。
4. 指令源副本执行 `transferTable`，向目标副本分块传输数据文件。
5. 传输完成后写入 `FINALIZING`，更新 `/assignments/{table}` 和 `/meta/tables/{table}`。
6. 确认删除源副本本地文件，恢复主副本写入，将状态改回 `ACTIVE`。
7. 任一步失败时恢复原元数据，并对目标残留执行确认删除。

迁移过程中，Client 收到 `MOVING` 后会按配置等待并重试。Master 还会在 `getTableLocation`、`listTables` 和 rebalance 前检查迁移状态；超过配置时间且补偿清理能够确认完成时，恢复为 `ACTIVE`，否则进入 `COMPENSATING` 并保留错误信息。

路由修复与故障恢复

当 RegionServer 下线时，Master 的 membership 监听器会触发定向 route repair。路由修复会检查主副本是否仍在线：

1. 如果主副本离线但存在在线副本，提升一个在线副本为新的 primary。
2. 如果副本数不足且存在可用 RegionServer，先冻结 clone source，调用 `transferTable` 补副本，再更新元数据。
3. 如果所有副本离线，将表状态标记为 `UNAVAILABLE`。

4. 当副本恢复在线后，路由修复逻辑会尝试把表恢复为 `ACTIVE`，并补齐目标副本数。

这部分实现避免了只更新元数据而没有数据的“假副本”。补副本必须先完成数据迁移，再写 ZooKeeper 元数据；迁移失败时系统会保持原副本状态并清理目标节点中的残留文件。

III.2. RegionServer 设计

Region Server，即从节点服务器，是系统数据面的核心。每个 RegionServer 启动一个 Java 服务进程，并管理本地 C++ MiniSQL 子进程。

启动与注册

RegionServer 启动时会读取 `RS_ID`、Thrift 端口、HTTP 端口、ZooKeeper 地址、WAL 目录、数据目录和 MiniSQL 可执行文件路径。启动后主要完成以下工作：

1. 连接 ZooKeeper，并在失败时持续重试，不以无 ZooKeeper 模式对外服务。
2. 启动 MiniSQL 进程。
3. 初始化 WAL 和副本同步状态。
4. 注册 Thrift 多路复用服务：`RegionService`、`RegionAdminService`、`ReplicaSyncService`。
5. 创建 `/region_servers/{rsId}` 临时节点。
6. 每 10 秒上报心跳、表数量、QPS、CPU、内存和副本决议信号。
7. 提供 HTTP `/health` 和 `/status` 端点。

MiniSQL 进程管理

Java 进程管理组件通过 `ProcessBuilder` 启动 C++ MiniSQL，并向其标准输入写入 SQL。它支持启动超时、执行超时、输出收集、崩溃后重启、显式 checkpoint 和停止。Java 分布式层不直接操作 C++ 内部结构，而是通过 SQL 和文件迁移边界与其交互。

SQL 执行路径

RegionServer 的 SQL 执行接口将 SQL 分为读和写两类。读操作直接交给 MiniSQL，跳过 WAL 和副本同步。写操作流程如下：

1. 检查 `WriteGuard`。如果表处于迁移冻结状态，返回 `MOVING`。
2. 通过 WAL 管理模块分配 LSN。
3. 将 SQL 作为 WAL payload，以 `PREPARE` 状态写入本地 WAL，并强制刷盘。
4. 从 `/assignments/{table}` 读取副本地址，排除自身。
5. 对普通 DML/索引 DDL 发起副本同步，默认等待至少 1 个从副本 ACK。
6. ACK 不足时把 WAL 标记为 `ABORTED`，返回错误。

7. ACK 满足要求后执行本地 MiniSQL。
8. 本地执行失败时标记 `ABORTED`，并向副本发送 ABORT 决议。
9. 本地执行成功后标记 `COMMITTED`，异步通知副本 commit，并触发副本追赶检查。

`CREATE TABLE` 和 `DROP TABLE` 是 Master 编排的 DDL，Master 已经对所有副本逐个发送 SQL，因此 RegionServer 写路径不会在这两类 DDL 中再次读取 assignment 做副本同步。

WAL 管理

WAL 管理模块为每张表维护一个 `{table}.wal` 文件。每条记录采用二进制格式：

```
[8B lsn][8B txnId][1B opType][1B status][4B sqlLen][sqlBytes]
```

状态分为 `PREPARE`、`COMMITTED`、`ABORTED`。恢复时只回放 `COMMITTED` 记录，避免处于 `PREPARE` 的写入被误执行。每次 append 后调用 `FileChannel.force(false)`，保证 ACK 前日志已经持久化。写操作达到阈值后触发 checkpoint，调用 MiniSQL 的 `checkpoint;` 刷新脏页，并压缩或删除旧 WAL。

副本同步

`ReplicaSyncService` 是从副本侧的 WAL 接收与回放服务。主副本调用 `syncLog` 后，从副本先将日志写入本地 WAL，再返回 ACK。主副本之后调用 `commitLog` 或 `finalizeLogDecision` 通知最终结果。

最终决议接口用于处理复杂故障：

1. `finalizeLogDecision(table, lsn, committed, decisionId, decidedAtMs)`：显式下发 COMMIT 或 ABORT。
2. `getLogDecisionState(table, lsn)`：查询某条日志是否已有最终决议。
3. 重复 COMMIT 和重复 ABORT 具有幂等语义。
4. 已 ABORT 的日志拒绝再被普通 COMMIT 覆盖，除非是可覆盖的超时自动 ABORT。

副本同步管理模块负责主副本侧协调。它并发调用副本 `syncLog`，等待达到 `requiredAcks`。commit 通知失败时进入待重试队列，并记录可疑副本、最终决议候选和统计指标。对于落后副本，系统会通过 `getMaxLsn + pullLog` 选择数据来源副本，按连续 LSN 回放缺失日志并补发 commit。

数据迁移与文件校验

Region 迁移使用管理 RPC 中的 `transferTable` 和 `copyTableData` 。源端先对 MiniSQL 做 checkpoint，保证磁盘文件可见，然后按 4096 字节分块传输给目标端。目标端先写 `.part` 临时文件，最后一块到达后再原子发布正式文件。

迁移完成后，源端发送 manifest 清单。目标端按文件名、大小、CRC32 和块签名校验。实现中还增加了路径安全检查、offset 连续性检查、重复 chunk 幂等确认、冲突重复包拒绝、空 manifest 拒绝、跨表文件拒绝和重复文件拒绝。这些检查保证迁移不会因为网络重试或异常输入写坏目标数据目录。

III.3. Client 设计

客户端 Client 是用户直接使用的交互层，同时承担路由缓存、重试和运行状态输出。

SQL 分类与路由

客户端支持以下命令类型：

类型	处理方式
`CREATE TABLE` / `DROP TABLE`	发送给 Active Master
`CREATE INDEX` / `DROP INDEX`	先解析表名，再转发到该表主副本 RegionServer
`SELECT` / `INSERT` / `DELETE`	按表名解析路由，访问 RegionServer
`SHOW TABLES`	调用 Master `listTables`
`execfile <path>`	解析脚本并顺序执行，连续同表 DML 使用 `executeBatch`
`SHOW ROUTING METRICS`	输出当前进程内路由统计

客户端暴露的 SQL 范围以 MiniSQL 已实现语句为主。Client 负责命令分类、路由和结果打印，具体 SQL 语义由 RegionServer 后端的 C++ MiniSQL 执行并返回；MiniSQL 未实现的 `UPDATE` 、`ALTER TABLE` 和 `TRUNCATE` 不作为已支持 SQL，测试中要求它们返回明确错误。

路由缓存与失效

路由缓存使用 `ConcurrentHashMap` 保存 `TableLocation` ，带 TTL 和版本号。默认 TTL 为 30 秒。Client 连接 ZooKeeper 后启动元数据监听机制，监听 `/meta/tables` 的 create/change/delete 事件，主动删除对应表的本地缓存。

当 RegionServer 返回 `REDIRECT`、`MOVING` 或 RPC 异常时，Client 会删除本地缓存并重新查询 Master。Master 返回的占位路由，例如 `TABLE_NOT_FOUND`、`ZK_UNAVAILABLE`、`NOT_LEADER` 或 `0.0.0.0:0`，不会写入缓存，避免错误路由长期停留在客户端。

重试与读降级

Client 支持 `MOVING` 有界重试或透明等待，重试间隔由环境变量控制。DML 在主副本短暂不可达时会删除缓存并重查路由。SELECT 在 `EVENTUAL` 模式下会按主副本、其余副本顺序尝试读取；在 `STRONG` 模式下只访问主副本。

路由指标

Client 记录按表的重定向次数、迁移重试次数、异常重试次数、路由回源次数和读降级次数。REPL 支持文本、JSON、Prometheus 和导出到文件。配置 `CLIENT_METRICS_HTTP_ENABLED=true` 后，Client 还会提供 `/metrics`、`/metrics/json` 和 `/healthz`。

III.4. C++ MiniSQL 存储引擎

最终存储引擎主要由以下模块组成：

模块	职责
Interpreter	解析用户输入 SQL，并调用 API 层执行
API	统一组织 Catalog、Record、Index 和 Buffer 操作
Catalog Manager	维护表、字段、主键、索引等元数据
Record Manager	管理定长记录文件，支持插入、删除和查找
Index Manager	实现 B+ 树索引创建、删除、等值查找和范围查找
Buffer Manager	管理页缓存、Clock 替换、pin/unpin、脏页刷新和 PageLSN
Log Manager	提供本地 WAL append、flush、redo recovery 和 LSN 管理

已支持的 SQL 包括：

- `create table`
- `drop table`
- `create index`
- `drop index`
- `select * from ...`
- `select * from ... where ...`
- `insert into ... values (...)`

- `delete from ...`
- `delete from ... where ...`
- `execfile <path>`

数据类型支持 `int`、`float` 和 `char(n)`。表定义支持主键和 unique 属性，索引基于 B+ 树。多条件查询已经支持 `AND` 和 `OR`。C++ 层提供 checkpoint 和 WAL crash recovery 能力，用于配合 Java 分布式层的持久化要求。

IV. 详细功能设计

IV.1. 建表流程

建表是典型的控制面流程。Client 将完整 DDL 发给 Active Master。Master 解析表名后查询在线 RegionServer 列表，按负载选择最多 3 个副本。随后 Master 对每个副本发送 `CREATE TABLE`，RegionServer 将 SQL 交给本地 MiniSQL 执行。只有所有目标副本都执行成功，Master 才写入 `/meta/tables/{table}` 和 `/assignments/{table}`。

该流程的关键点是“先创建数据副本，后发布元数据”。如果先写元数据，Client 可能查到一个还没有数据文件的路由；本系统通过先执行副本 DDL、再写 ZooKeeper 元数据的顺序避免这种状态。如果元数据写入失败，Master 会删除已创建副本；如果某个副本建表失败，Master 会删除此前成功的副本。

IV.2. 写入流程

普通写入包含 DML 和索引 DDL。流程如下：

图 2 — 写入确认流程图

图 2 展示了写入确认顺序。主副本不会在 ACK 不足时执行本地 MiniSQL，因此客户端收到失败时不会对应一个已经在主副本提交的写入。

1. Client 根据表名从缓存读取路由，缓存未命中时访问 Master。
2. Client 直连主副本 RegionServer 调用 `RegionService.execute`。
3. RegionServer 检查迁移写冻结。
4. 主副本写本地 WAL `PREPARE` 并刷盘。
5. 主副本并发调用从副本 `ReplicaSyncService.syncLog`。
6. 至少 1 个从副本 ACK 后，主副本执行本地 MiniSQL。
7. 本地执行成功后，主副本提交 WAL，并异步通知从副本 COMMIT。
8. 从副本执行 `commitLog` 时回放 WAL 中保存的 SQL。
9. 若部分从副本落后，由 `pullLog` 追赶。

如果 ACK 不足，主副本把本地 WAL 标记为 `ABORTED` 并拒绝执行 SQL。这是系统 CP 取舍的直接体现。

IV.3. 查询流程

查询不经过 Master 的数据转发。Client 获取路由后直接访问 RegionServer。默认情况下，Client 优先访问 primary；若配置为 `EVENTUAL`，主副本失败时按副本列表尝试其他副本。若配置为 `STRONG`，只访问 primary。

查询结果由输出解析模块将 MiniSQL 标准输出解析为 `QueryResult`，再返回给 Client 打印。端到端测试覆盖了等值查询、范围查询、`AND/OR` 条件、删除后查询、索引创建删除和脚本执行。

IV.4. Master 故障转移

Master 通过 ZooKeeper 保证单主。Active Master 停止后，LeaderLatch 会选出新的 Active Master，新主通过 CAS 更新 `/active-master` 的 `epoch`。Client 在读取 `/active-master` 时优先使用 `address` 字段；若连接到 Standby Master，则通过 `NOT_LEADER` 响应中的 `redirectTo` 重新访问 Active Master。

混沌测试验证了 Active Master 停止后 `epoch` 递增，客户端可以通过新的 Active Master 继续访问集群。网络隔离场景下，系统保持单主语义，恢复后原 Master 变为 Standby。

IV.5. RegionServer 故障与恢复

RegionServer 通过 ZooKeeper 临时节点注册。节点停止或失联后，其临时节点被删除，Master 的 Watcher 感知到 membership 变化。Master 会扫描受影响表：

1. 若 primary 离线且存在在线副本，则提升在线副本为 primary。
2. 若副本数低于目标值且有新节点可用，则从存活副本迁移文件补副本。
3. 若没有在线副本，则将表状态置为 `UNAVAILABLE`。
4. 恢复后再次扫描，把表状态改回 `ACTIVE` 并补齐副本。

在 CP 契约下，故障期间写入可能失败，但必须返回明确错误；若写入返回成功，恢复后必须能读到对应数据。最终混沌测试按这一契约验证。

IV.6. Region 迁移流程

迁移用于 rebalance 和补副本。核心步骤是冻结写入、传输文件、校验 manifest 清单、更新元数据、清理旧文件。Client 在迁移期间收到 `MOVING`，按等待策略重试。

图 3 — Region 迁移状态图

图 3 展示了迁移的主要状态。正常路径会回到 `ACTIVE`，失败路径保留恢复所需的上下文，直到元数据和目标残留文件处理完成。

本系统特别强调迁移安全性。目标端不会直接覆盖正式文件，而是先写 staging 文件；manifest 清单校验失败时不会确认迁移完成。Master 在失败路径上恢复原始元数据，并确认清理目标端残留。这样可以避免“路由已更新但数据没传完”的错误状态。

V. 关键实现难点与处理方式

V.1. 元数据发布顺序

分布式建表的难点不在于单个节点执行 `CREATE TABLE`，而在于何时向客户端发布路由。如果先写 ZooKeeper 元数据，客户端可能查到一个尚未在 RegionServer 上创建完成的表；如果只在部分副本成功后发布元数据，后续写入又会遇到副本集合不完整的问题。

本系统采用“先副本、后元数据”的顺序。Active Master 先选择目标 RegionServer，并逐个下发建表 DDL。只有目标副本均执行成功后，才写入表路由和 assignment。若中间任一副本失败，Master 会删除此前已创建的副本；若元数据写入失败，也会撤销已创建的副本。这样客户端看到的表路由对应已经准备好的数据副本。

删表流程采用相反方向的谨慎策略。Master 先对 assignment 中的副本执行 `DROP TABLE`，只有副本返回成功或表不存在后才删除 ZooKeeper 元数据。若存在失败副本，元数据继续保留，使后续请求仍能知道这张表处于需要继续处理的状态。

V.2. 写入确认顺序

写入路径需要同时处理本地执行、副本同步和故障返回。如果主副本先执行 MiniSQL 再同步副本，一旦副本 ACK 不足，系统就会进入“客户端失败但主副本已经写入”的状态。为避免这种状态，RegionServer 的写入顺序是 WAL `PREPARE`、从副本同步、主副本执行、WAL `COMMITTED`、从副本提交。

在三副本部署下，主副本默认等待至少一个从副本 ACK。ACK 不足时，主副本把本地 WAL 标记为 `ABORTED` 并直接返回错误，不执行 MiniSQL 写入。这个顺序牺牲了故障期间的一部分可用性，但保持了 CP 设计中对提交结果的约束：写入要么有多数确认后提交，要么以明确错误返回。

V.3. 迁移期间的数据可见性

Region 迁移需要处理两个边界：一是迁移过程中不能继续产生新的写入，二是目标副本不能在文件未完整传输时被加入路由。系统在迁移前把表状态推进到 `PREPARING`，随后暂停主副本写入，再进入 `MOVING` 状态。Client 收到 `MOVING` 后按配置等待和重试，不把迁移中的路由当作正常可写路由。

数据传输采用分块复制。源端在传输前执行 checkpoint，使 MiniSQL 中的脏页写到磁盘；目标端先写 `.part` 临时文件，最后一块到达后再发布正式文件。传输完成后，目

标端根据 manifest 校验文件名、大小、CRC32 和块签名。只有校验通过后，Master 才更新 assignment 和表元数据。

V.4. 客户端路由失效处理

客户端缓存能减少 Master 查询压力，但缓存错误路由会放大故障影响。系统对路由缓存设置了 TTL 和版本号，并监听表元数据变化。一旦表元数据创建、修改或删除，Client 会主动删除本地缓存。RegionServer 返回 `REDIRECT`、`MOVING` 或 RPC 异常时，Client 也会清除对应表缓存并重新访问 Master。

Master 在部分异常场景下会返回占位路由，例如表不存在、ZooKeeper 不可用、当前节点不是 Active Master，或地址为 `0.0.0.0:0`。Client 不会缓存这些占位结果，而是把它们作为临时不可用状态处理。这样可以避免一次短暂的元数据不可见，变成后续多次请求持续访问错误地址。

V.5. 测试对实现边界的确认

最终测试不仅覆盖正常 SQL 路径，也覆盖实现边界。MiniSQL 未实现的 `UPDATE`、`ALTER TABLE` 和 `TRUNCATE` 被测试为明确错误返回，而不是作为已支持功能静默通过。混沌测试也不要求故障期间写入一定成功，而是按 CP 契约检查：成功提交的数据在恢复后可见，失败请求不能产生脏提交。

这种测试方式与系统设计保持一致。系统不追求在任意分区下继续写入，而是优先保持元数据、路由和副本状态的一致性；当副本确认不足、迁移尚未完成或主节点尚未切换完成时，请求可以失败，但必须给出明确状态。

VI. 系统测试与验证

VI.1. 自动化测试体系

本项目建立了分层测试体系，覆盖 C++ 引擎、Java 模块、Docker 集群、ZooKeeper 拓扑、端到端 SQL 和混沌场景。

测试层级	覆盖内容
C++ 单元测试	基础数据结构、Buffer、Catalog、Index、Record、API
C++ WAL	WAL append、flush、LSN 单调、崩溃 redo
C++ 压力测试	4000+ 插入、删除、索引重建、pin 计数
Java 单测	Master、RegionServer、Client 的组件级测试
Java 集成测试	内嵌 ZooKeeper 下的选主、注册、元数据和路由测试
集群冒烟	Docker 集群下 DDL、DML、Index、WHERE、DELETE、错误返回
集群压力	并发写、大表导入、范围查询和索引压力
ZK 拓扑	ZNode 结构、Active Master、RegionServer 注册和表元数据
端到端 Client	SQL 测试用例与期望输出对照
混沌测试	Master 故障转移、RS 崩溃、随机 RS 崩溃、网络分区、可疑副本

VI.2. 系统测试结果

根据项目测试记录，主要测试结果如下：

套件	结果	说明
`cpp.unit`	PASS	C++ 基础模块测试通过
`cpp.wal`	PASS	WAL 与 crash recovery 测试通过
`cpp.stress`	PASS	压力测试通过
`java.unit`	PASS	Java 单测 304 项通过
`java.integration`	PASS	Java 集成测试 80 项通过
`cluster.smoke`	PASS	Docker 集群基础 SQL 流程通过
`zk.topology`	PASS	ZooKeeper 拓扑断言通过
`cluster.stress`	PASS	2×50 并发写、500 行导入和 200 行索引范围查询通过
`e2e.client`	PASS	干净环境下 7/7 SQL 测试用例通过

cluster.chaos	PASS	5/5 场景通过，按 CP 契约验证

混沌测试中的 RegionServer 故障场景按 CP 契约检查：写请求要么成功提交并在恢复后可见，要么返回明确错误且不能产生脏提交。这个验收标准与系统的一致性取舍一致。

VI.3. 验证结论

自动化测试覆盖了存储引擎、分布式控制面、数据面、客户端路由、容器化部署和故障场景。最终验证结果表明，系统可以完成从建表、写入、查询、索引、删除到脚本执行的端到端流程；在 Master 切换、RegionServer 停止、随机故障和网络分区场景下，系统按 CP 契约保持元数据和数据状态一致。

VII. 系统功能实现情况

VII.1. 核心功能

功能	状态	实现说明
ZooKeeper 集群管理	完成	三节点 ZooKeeper，保存 Master、RS、表元数据与 assignment
多 Master 高可用	完成	Curator LeaderLatch 选主，`/active-master` 保存 epoch 与地址，非主返回重定向
RegionServer 注册与心跳	完成	临时节点注册，每 10 秒上报负载、状态和副本决议信号
表级数据分布	完成	每张表作为一个 Region，按表分配到多个 RegionServer
三副本策略	完成	建表时选择最多 3 个副本，RegionServer 故障后由路由修复和迁移流程补齐副本
半同步写入	完成	主副本写 WAL，等待默认 1 个从副本 ACK，再执行本地写入
WAL 与恢复	完成	Java WAL 支持 PREPARE/COMMITTED/ABORTED，C++ 层支持本地 WAL crash recovery
负载均衡	完成	基于 tableCount、QPS、CPU、内存评分，定时或手动触发迁移
Region 迁移	完成	写冻结、文件分块传输、manifest 校验、元数据更新、清理确认
容错容灾	完成	RS 下线触发 route repair，主副本晋升，补副本，UNAVAILABLE 状态
客户端缓存	完成	TTL、版本号、ZK Watcher 主动失效、REDIRECT/MOVING/异常后重查

分布式查询	完成	Client 直连 RegionServer 查询，支持按配置读降级
容器化部署	完成	Docker Compose 启动 3 ZK + 3 Master + 3 RS + Client
混沌测试	完成	覆盖 Master failover、RS crash、随机 RS crash、网络分区和可疑副本

VII.2. 工程质量与测试保障

本系统不仅实现正常执行路径，也对多类异常分支进行了处理：非主 Master 拒绝写控制请求，建表元数据失败时回滚已创建副本，删表只在所有副本确认后删除元数据，迁移失败恢复原 assignment，副本 commit 失败进入重试队列，Client 不缓存无效路由，RegionServer 文件迁移校验 manifest 清单，DROP TABLE 后清理孤儿文件。

测试文档记录了各层测试的执行结果，覆盖 C++ 存储引擎、Java 分布式层、Docker 集群和混沌测试。项目记录中的核心套件为本系统主要行为提供了可复现的验证依据。

VIII. 总结

本项目最终实现了一个可部署、可执行、可测试的分布式 MiniSQL 系统。系统使用 C++ MiniSQL 提供单机数据库能力，使用 Java 17、ZooKeeper 和 Thrift 搭建分布式控制面与数据面，完成了表级分布、三副本、半同步写入、Master 高可用、RegionServer 注册心跳、路由缓存、迁移、路由修复、WAL、故障恢复和混沌测试。

本项目的主要价值在于把课程中的分布式系统概念实现为可运行工程：选主需要处理旧主恢复和重定向，副本同步需要处理 ACK 不足和 commit 通知丢失，迁移需要处理文件不完整和元数据更新顺序，客户端缓存需要处理失效与错误路由，测试需要覆盖节点停止、网络隔离和脏状态恢复。

在课程项目范围内，SuperSQL 已经实现了分布式数据库的核心链路，并通过自动化测试和混沌测试验证了主要行为。